

RAPPORT 3 — Traitement du Langage Naturel : Analyse Comparative RNN, LSTM, GRU et Architecture Seq2Seq

Projet : Deep Learning — Partie 3 Date : Juin 2026

1. Introduction et Objectifs

Ce rapport est dédié à l'étude approfondie des **Réseaux de Neurones Récurrents (RNN)** appliqués au Traitement du Langage Naturel (NLP). Le périmètre expérimental couvre deux tâches fondamentales et hautement complémentaires :

1. **Génération de texte** caractère par caractère (évaluation comparative des architectures RNN, LSTM et GRU).
2. **Traduction automatique** de phrases s'appuyant sur une architecture Seq2Seq (Encoder-Decoder).

Objectifs scientifiques : Cerner précisément les limites intrinsèques du RNN simple (phénomène de vanishing gradient), quantifier l'apport structurel des mécanismes de portes (gating), et implémenter un pipeline de traduction complet doté de stratégies de décodage avancées.

2. Méthodologie

2.1 Tâche 1 : Génération de Texte Caractère par Caractère

Dataset exploité

- **Corpus** : Un extrait choisi de la pièce *Roméo et Juliette* de William Shakespeare.
- **Volume** : 625 caractères.
- **Vocabulaire unique** : 41 caractères distinctes.
- **Formatage séquentiel** : Séquence d'entrée (input) fixed à 10 caractères → prédiction du caractère immédiatement suivant (cible).

Architectures Comparées

Développement d'une classe PyTorch générique nommée `CharRNN` encapsulant dynamiquement l'argument `rnn_type` :

Modèle	Cellule PyTorch	Mécanismes de Portes (Gates)	Dimension cachée
RNN simple	<code>nn.RNN</code>	Aucune (récurrence linéaire simple)	64
LSTM	<code>nn.LSTM</code>	Input Gate, Forget Gate, Output Gate	64
GRU	<code>nn.GRU</code>	Reset Gate, Update Gate	64

Architecture commune partagée :

Embedding(41→64) → RNN/LSTM/GRU(64) → Linear(64→41) → Softmax

Stratégie d'Entraînement

- **Optimiseur** : Adam.
- **Fonction de Coût (Loss)** : CrossEntropyLoss.
- **Gradient Clipping** : Fixé rigoureusement à `max_norm=5.0` pour endiguer l'explosion des gradients induite par la BPTT (Backpropagation Through Time).
- **Nombre d'épochs** : 20.

Learning rates différenciés (après phase d'ajustement) :

- RNN simple : $lr = 0.05$ (requis pour pallier la lenteur de convergence due au vanishing gradient).
- LSTM / GRU : $lr = 0.005$ (les portes stabilisant nativement la propagation du gradient, un lr plus faible est optimal).

2.2 Tâche 2 : Traduction Automatique Seq2Seq

Dataset

Un corpus ciblé de 20 paires de phrases Français ↔ Anglais, générant un vocabulaire global de 97 mots uniques intégrant les tokens indispensables `<SOS>`, `<EOS>`, et `<UNK>`.

Architecture de l'infrastructure Encoder-Decoder

EncoderRNN : `Embedding → GRU(64) → Context Vector (Hidden state final de la séquence)`

DecoderRNN : `Embedding → GRU(64, initialisé avec le Context Vector) → Linear(64→vocab_size) → Softmax`

Techniques Algorithmiques Avancées Implémentées

Technique	Rôle et impact fonctionnel
Teacher Forcing (ratio=0.5)	Alimente le décodeur avec 50% de mots cibles réels et 50% de ses propres prédictions précédentes pour accélérer la convergence initiale.
Gradient Clipping	Stabilisation mathématique indispensable du flux récurrent.
Greedy Search	Sélection systématique et exclusive du mot le plus probable à chaque pas de temps.
Beam Search (largeur k=3)	Exploration simultanée des 3 hypothèses de phrases les plus probables, limitant les erreurs locales.

Évaluation Métrique : Score BLEU-n

$$BLEU = BP \cdot \exp(\sum_{i=1}^n \log p_i / n)$$

Implémentation avec **smoothing de type epsilon** (substitution des valeurs nulles par 1e-8) et application de la **Brevity Penalty (BP)** pour éviter de favoriser indûment les traductions incomplètes.

3. Résultats Expérimentaux

3.1 Génération de Texte : Comparaison RNN / LSTM / GRU

Modèle	Loss Finale	Perplexité (PPL)	Dynamique de Convergence
RNN simple	0.2351	1.27 	Extrêmement rapide avec lr adapté.
GRU	1.1261	3.08	Stable mais plus lente.
LSTM	1.4032	4.07	Stable mais sous-optimale ici.

Évolution pas-à-pas des métriques pour l'architecture RNN :

- Epoch 5 : Loss = 1.0412, Perplexité = 2.83
- Epoch 10 : Loss = 0.5864, Perplexité = 1.80
- Epoch 15 : Loss = 0.3703, Perplexité = 1.45
- Epoch 20 : Loss = 0.2351, Perplexité = 1.27

3.2 Traduction Seq2Seq

- **Loss finale** : 0.1323 atteinte au terme de 30 epochs.
- **Convergence** : Nettement accélérée par le mécanisme de Teacher Forcing.
- **Qualité linguistique** : L'algorithme de Beam Search surpasse qualitativement le Greedy Search en termes de cohérence syntaxique.

4. Analyse et Discussion

4.1 Pourquoi le RNN simple surpasse-t-il le LSTM/GRU dans ce cas précis ?

Ce résultat de prime abord **contre-intuitif** trouve sa justification mathématique dans la nature même de la tâche :

1. **Séquences extrêmement courtes (10 caractères)** : Le problème du s'évanouissement du gradient (vanishing gradient) ne se manifeste pas à cette échelle.
2. **Espace de recherche restreint** : Le vocabulaire de 41 caractères limite drastiquement la complexité topologique de la loss.
3. **Sur-capacité (Overfitting)** : La richesse structurelle des portes des LSTM/GRU provoque un sur-apprentissage immédiat sur un mini-corpus de 625 caractères.

4.2 Rôle du Gradient Clipping

Le fait de plafonner la norme des gradients à `5.0` élimine radicalement les instabilités numériques lors de la rétropropagation à travers le temps (BPTT), garantissant des mises à jour douces des poids synaptiques.

4.3 Greedy vs Beam Search : Le compromis algorithmique

Méthode de décodage	Avantage majeur	Inconvénient majeur
Greedy Search	Complexité temporelle minimale, exécution ultra-rapide.	Sensible aux choix initiaux erronés, pas de retour arrière.
Beam Search (k=3)	Optimisation globale de la phrase, meilleure fluidité.	Coût mémoriel et calculatoire multiplié par k.